

Informe final

Semana: 9

Nombre del estudiante:

Marco Vásquez

Número de cuenta:

22541183

Sede de estudio:

Unitec San Pedro Sula

Docente:

MSc. Paul Idiáquez

Sección:

39

Fecha de entrega:

18/9/26

Contenido

Introducción	4
Justificación	4
Objetivos.....	4
Descripción general del proyecto	5
Funcionalidades disponibles para el usuario	5
Clases y responsabilidades de cada una.....	5
Relación entre las clases.....	6
Diagramas de Lógica.....	6
Investigación realizada.....	8
Cifrado y descifrado César	8
Cifrado y descifrado XOR.....	9
Cifrado y descifrado Vigenère.....	10
Generalización a bytes (para archivos e imágenes)	11
Elementos de investigación propia	12
1. Librería <ctime>	12
2. Selección de archivos mediante funciones nativas del sistema	12
3. Personalización de consola	12
4. Manejo de archivos binarios	13
5. Compilación estática (static)	13
6. Funciones virtuales	13
Manual Técnico	14
Arquitectura de clases y principios de POO	14
Abstracción (clase Cifrador).....	14
Uso de modificador protected	14
Herencia y encapsulamiento.....	15
Polimorfismo.....	15
Uso técnico de punteros dentro del sistema.....	15
1. Buffer para procesamiento de archivos (unsigned char* buffer)	15
2. Puntero polimórfico (Cifrador* miCifrador).....	15
3. Doble puntero (Usuario** listaUsuarios)	16
Optimización de memoria	16
1. Arreglos dinámicos “inteligentes”	16

2. Reserva exacta de bytes	16
Módulo de historial, auditoría y ordenamiento (Nivel 4)	16
Entidad Operacion.....	17
Algoritmo de ordenamiento por fecha (Burbuja).....	17
Implementación de búsqueda lineal.....	17
Pruebas y resultados	18
Pruebas con datos inválidos	18
Prueba con datos válidos	19
Evidencia de validaciones y del historial.....	19
Resultado final obtenido.....	20
Manual de Usuario	20
Introducción y propósito del sistema	20
Requisitos de ejecución y guía de inicio rápido.....	20
Guía de pasos de inicio rápido.....	20
Control de acceso y gestión de usuarios	21
Operaciones en consola y menú principal (opciones 1-6)	22
Operaciones de archivos y auditoría (opciones 7 a 9).....	23
Opción7: Copiar archivo binario	23
Opción 8: Cifrar/Descifrar archivo.....	23
Opción 9: Historial y reportes (submenú).....	23
Solución de problemas	24
Conclusiones	25
Bibliografías/Referencias	25
Anexos	26

Introducción

El proyecto presentado se trata de un Cripto-Manager, el cuál es un sistema de cifrado y descifrado de archivos multinivel desarrollado en C++. El sistema aborda la necesidad de asegurar información confidencial procesando tantos textos planos en consola como archivos binarios completos de cualquier extensión, para que nadie que no posea la clave ni sepa el algoritmo correspondiente pueda ver el contenido ni corromperlos. Las tecnologías principales utilizadas fueron el manejo correcto de memoria dinámica, la manipulación de archivos binarios, la implementación de funciones específicas del sistema operativo para la selección de archivos y personalización de consola, utilizando la API de Windows en este entorno presentado (.exe) y alternativas multiplataforma para otros sistemas. En este informe el lector podrá encontrar la explicación detallada del funcionamiento del sistema, la lógica matemática de los algoritmos que se implementaron y las técnicas de gestión de memoria dinámica aplicadas.

Justificación

El proyecto fue desarrollado para demostrar y poner en práctica los elementos aprendidos en clase y los elementos investigados por cuenta propia. También, fue desarrollado para otorgar a los usuarios una herramienta capaz de proteger datos sensibles de su elección. El desarrollo de este permite aplicar los conocimientos brindados como Programación Orientada a Objetos (clase abstracta y polimorfismo), el uso de punteros dobles y simples, y la correcta administración de arreglos dinámicos en el Heap. La importancia de las técnicas aplicadas radica en separar el programa en módulos e interfaces que facilitan la escalabilidad a futuro.

Objetivos

Objetivo general: Desarrollar un programa en C++ para el cifrado/descifrado de archivos y textos en consola, aplicando una gestión de usuarios y un historial de operaciones, respaldado por la correcta administración de memoria dinámica y abstracción de objetos.

Objetivos específicos:

- Implementar los algoritmos criptográficos César, XOR y Vigenère, generalizados para trabajar a nivel de bytes (0-255) para manipular archivos binarios.
- Mejorar la experiencia de usuario implementando la selección de archivos mediante el explorador nativo del sistema operativo cuando esté disponible.
- Generar un historial que registre las operaciones con fecha y hora, además del algoritmo que se utilizó y con qué tipo de operación.
- Administrar arreglos dinámicos y buffers de lectura para procesar de forma segura archivos binarios, implementando funciones de redimensionamiento automático.

Descripción general del proyecto

El sistema cuenta con un módulo GestorUsuarios que valida las credenciales de cada usuario al momento del inicio de sesión. Si el acceso es exitoso, se despliega un menú principal con las 10 opciones posibles de control del sistema, mostrando con colores los textos de error, salidas esperadas y títulos del sistema.

Sus características principales son la interfaz de consola con soporte UTF-8 (para tildes y letra ñ), retroalimentación visual durante ejecución, invocación del explorador nativo del sistema operativo para la selección de archivos. El proyecto fue desarrollado en base al cumplimiento de 4 niveles:

Nivel	Descripción
Nivel 1	Cifrado y Descifrado César por desplazamiento dinámico sobre texto y archivos .txt
Nivel 2	Cifrado XOR mediante clave secreta ingresada por el usuario con manejo de bloques de datos.
Nivel 3	Cifrado Vigenère o algoritmo por sustitución/transposición poli alfabética aplicado a archivos de texto y lectura binaria.
Nivel 4	Sistema completo de administración de usuarios (Login/Roles) almacenado en archivo, historial dinámico de operaciones con fecha/hora, generación de reportes y estadísticas de uso.

Funcionalidades disponibles para el usuario

El usuario dispone al ejecutar el programa son: Iniciar sesión, registrarse, encriptar y desencriptar mensajes en consola, copiar archivos binarios (validar que no haya problemas con archivos), cifrar/descifrar archivos mediante el explorador nativo del sistema operativo, ver el historial completo tabulado, ordenar el historial de manera cronológica, buscar operaciones por filtros de nombre (nombre de usuario) y generar reporte de estadística de uso de los algoritmos.

Clases y responsabilidades de cada una

Clase	Responsabilidad
Cifrador	Clase abstracta que define la interfaz común (cifrar, descifrar, cifrarByte, descifrarByte) mediante funciones virtuales puras.
CifradorCesar	Hereda de Cifrador. Implementa el cifrado por desplazamiento en su versión de consola y de bytes para archivos.
CifradorXOR	Hereda de Cifrador. Implementa el cifrado bit a bit con clave única.
CifradorVigenere	Hereda de Cifrador. Implementa la sustitución polialfabética en su versión de letras y bytes.
Utilidades	Funciones estáticas de apoyo para validaciones, conversión a hexadecimal, manejo de consola (limpiar, pausar y color), explorador nativo de archivos, fecha/hora y el manejo de archivos binarios.
GestorArchivos	Cifra/descifra un archivo completo (de texto o binario) usando polimorfismo, no conoce el algoritmo específico que se está usando.
Usuario	Encapsula el nombre de usuario, contraseña y rol.
GestorUsuarios	Administra un arreglo dinámico de usuarios mediante un puntero doble (Usuario**), con registro, login, búsqueda y persistencia en archivo.

Operación	Encapsula el registro del historial, guardando datos como: usuario, tipo de operación, algoritmo, archivo y su respectiva fecha/hora.
HistorialOperaciones	Administra un arreglo dinámico de Operación, con registro, ordenamiento por fecha (burbuja), búsqueda por usuario (lineal), generación de reportes y persistencia en archivo.

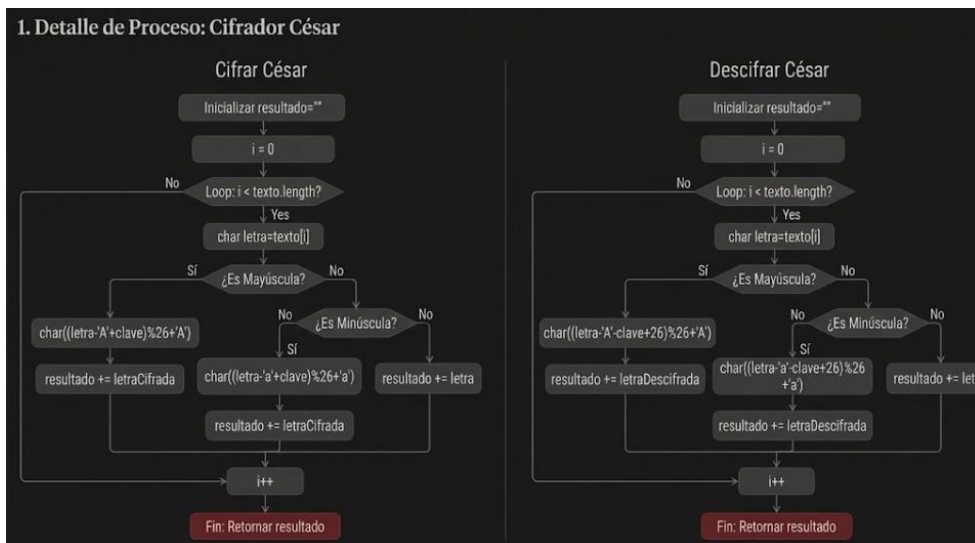
Relación entre las clases

CifradorCesar, CifradorXOR y CifradorVigenere heredan públicamente de la clase Cifrador. GestorArchivos y el main utilizan un puntero Cifrador* para trabajar con cualquiera de los tres algoritmos de forma polimórfica (puntero polimórfico). GestorUsuarios administra internamente varios objetos de tipo Usuario mediante un doble puntero (Usuario**). De igual manera, HistorialOperaciones administra un arreglo de objetos de tipo Operación. La clase main.cpp actúa como un controlador de flujo, es la encargada de delegar toda la lógica entre estas clases.

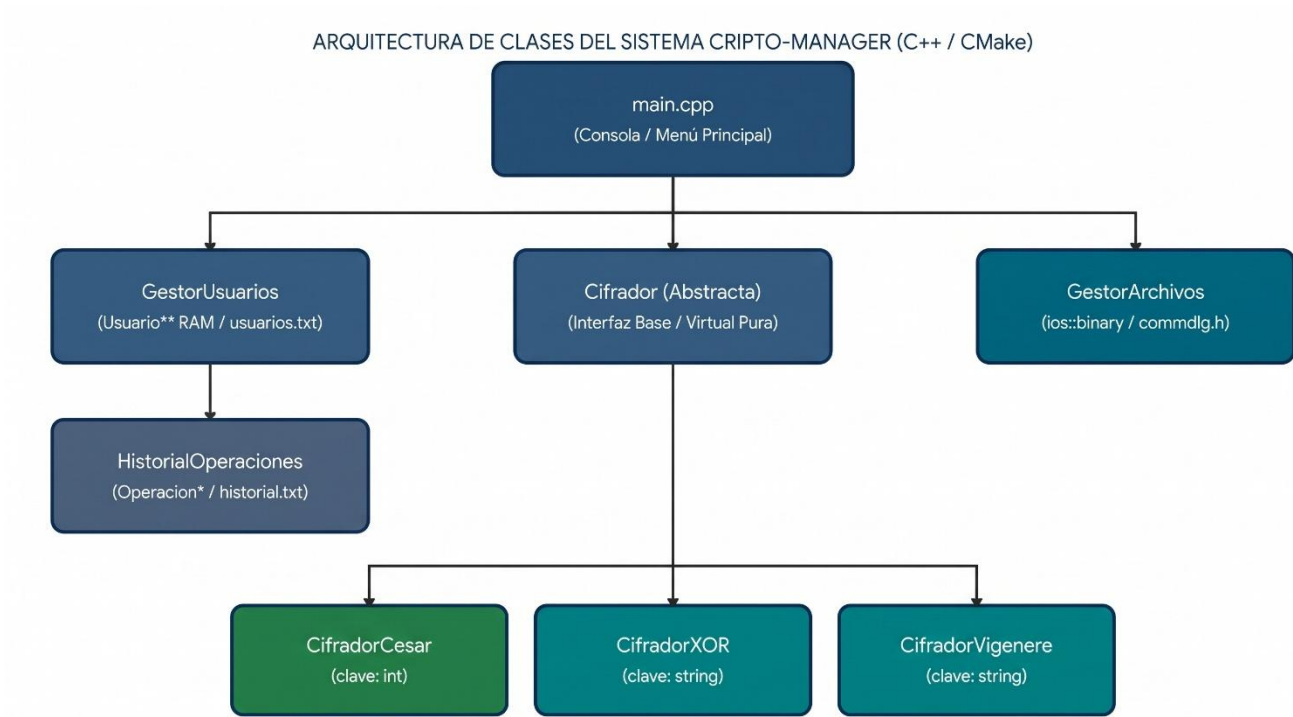
Diagramas de Lógica

A continuación, se presentan diagramas de lógica que sirven como una ayuda visual para el entendimiento del sistema y de sus algoritmos.

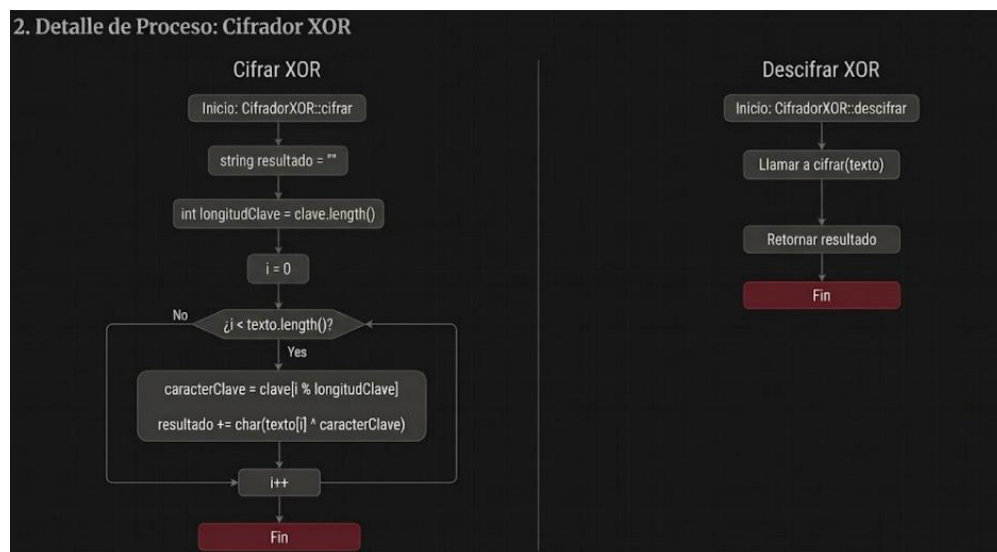
El siguiente diagrama ilustra la lógica detrás del cifrado César mediante dos ruedas: la exterior representa el alfabeto original y la interior el alfabeto desplazado según la clave elegida. Y también la lógica en código.



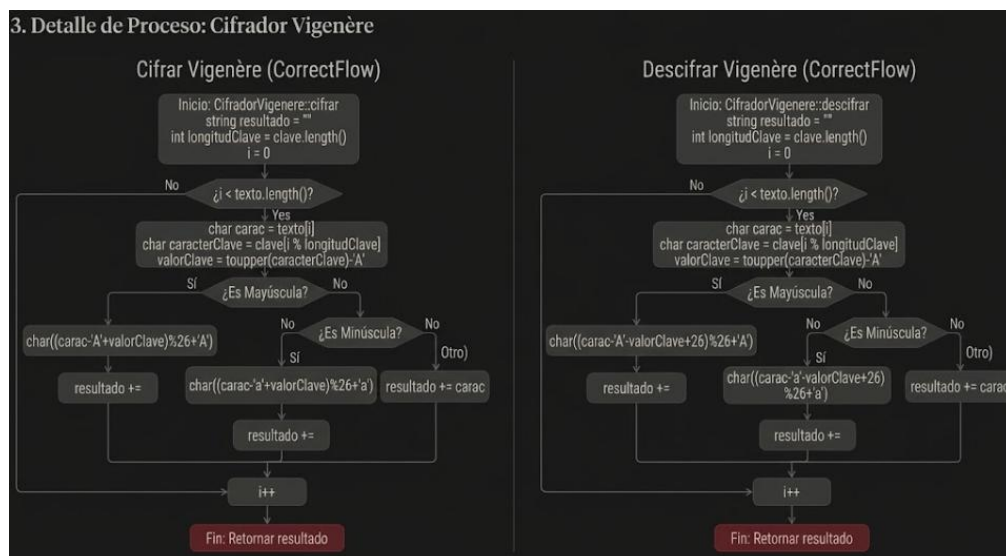
El siguiente diagrama resume el flujo general del sistema: desde el ingreso del usuario, pasando por la selección del algoritmo y la acción a realizar, hasta la lectura, transformación byte a byte y la escritura del archivo resultante.



El siguiente diagrama explica de forma visual como funciona el cifrado XOR normal con texto plano y con archivos. El cifrado XOR funciona igual para ambos ya que trabaja bit a bit.



El siguiente diagrama visual explica el funcionamiento normal del cifrado Vigenère con textos planos. La palabra al ser más larga que la contraseña, vuelve a repetirse hasta llenarla.

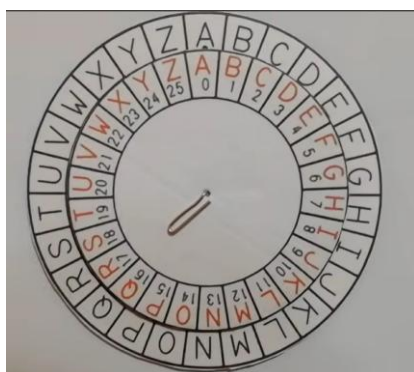


Investigación realizada

A continuación, se documentan los algoritmos investigados para el correcto funcionamiento del sistema, así como los elementos investigados por cuenta propia que no fueron desarrollados en su totalidad dentro del marco teórico de la clase.

Cifrado y descifrado César

El cifrado César consta de una clave (entre 1 – 26 para el alfabeto inglés y del 1 – 27 para el alfabeto español) para desplazar el abecedario esa cantidad de posiciones. Podemos tener una clave 3 (espacios) y nuestro abecedario se desplaza esa cantidad a la derecha para obtener una letra ya cifrada. Un ejemplo, siempre con clave 3, la letra A pasa a ser la letra D, la B pasa a la letra E, la C para a la letra F y así sucesivamente con el resto del alfabeto. Si el número de desplazamiento llega al final del abecedario y ya no hay más letras, las letras que sobran siguen avanzando las posiciones que deben, pero el inicio del abecedario. Ejemplo: la letra Z (clave 3) pasaría a ser C. La imagen de abajo es un sistema visual de cómo hacer el cifrado/descifrado César (moviendo el círculo interior las posiciones deseadas).



Este tipo de cifrado debe de tener una clave previamente acordada por ambas partes, emisor y receptor. En el caso del programa el usuario ingresa la clave.

Ejemplo más práctico/visual

Mensaje: Hola mundo

Clave: 3

Mensaje cifrado: Krod pxqgr (hola mundo ya cifrado)

Para lograr cifrar y descifrar los mensajes existe dos fórmulas matemáticas.

$C=(X+K) \bmod N$ (formula cifrado) y $X=(C-K) \bmod N$ donde:

C -> posición de la letra cifrada resultante

X -> posición de la letra original

K -> clave de desplazamiento

N -> número total de letras del alfabeto (26 inglés, 27 español)

A nivel de código:

- Restamos 'A' para llevar el valor de la letra a un rango de 0 a 25 para que la operación del residuo (% 26) funcione correctamente.
- Sumamos 'A' al final para regresar el resultado al rango original de la tabla ASCII (65 a 90) y que la pantalla muestre una letra real.
- Se hace lo mismo con 'a' (97) para que las minúsculas no se mezclen ni se transformen en mayúsculas.

Cifrado y descifrado XOR

Este tipo de cifrado trabaja con bits (unos y ceros) y su lógica radica en comparar dos bits:

Caso 1: si ambos son iguales (0 y 0 o 1 y 1) el resultado es 0

Caso 2: si son diferentes (0 y 1 o 1 y 0) el resultado es 1

Para cifrar: Mensaje(+)Clave=mensajeCifrado

Para descifrar: MensajeCifrado(+)Clave=mensajeOriginal

Donde el (+) es el símbolo utilizado para el algoritmo llamado suma encirculada.

Para este cifrado se utiliza una clave (que puede ser un número, una letra o una palabra). Si la clave es más corta que el mensaje, esta se repite cíclicamente carácter por carácter hasta cubrir la totalidad del texto. Es un cifrado extremadamente rápido debido a que la computadora realiza la operación bit a bit de forma directa.

```

char CifradorXOR::cifrarByte(unsigned char byte,int posicion){
    unsigned char byteClave=(unsigned char)clave[posicion%clave.length()];
    return (char)(byte^byteClave);
}
char CifradorXOR::descifrarByte(unsigned char byte, int posicion){
    return cifrarByte(byte,posicion);
}

```

El código del XOR al trabajar con bytes, como mencioné antes, es una función corta. Se hace el casteo unsigned char para evitar valores negativos. Para descifrar XOR solo se debe de volver a utilizar la función cifrarByte, ya que hacerlo dos veces devuelve el mensaje original.

Cifrado y descifrado Vigenère

La lógica de este cifrado radica en repetir una palabra clave a lo largo de todo el mensaje que se desea cifrar. Cada letra de la clave indica cuantas posiciones se debe desplazar la letra correspondiente del mensaje original usando el valor numérico de esa letra (A=0,B=1,C=2, etc).

Al igual que el sistema XOR si la clave es más corta que el mensaje este se repite cíclicamente hasta cubrir todo el texto.

Para cifrar: Se suma la posición de la letra del mensaje con la posición de la letra de la clave

Para descifrar: Se resta la posición de la letra de la clave a la letra del mensaje cifrado

Contamos con dos fórmulas matemáticas que sirven como guía:

Cifrar: $(\text{letra mensaje} + \text{letra clave}) \% N$

Descifrar: $(\text{letra mensaje} - \text{letra clave} + N) \% N$

Donde N -> número total de letras del alfabeto que se esté utilizando (inglés o español)

```

string CifradorVigenere::cifrar(string texto){
    string resultado="";
    int longitudClave=clave.length();
    for(int i=0;i<texto.length();i++){
        char letra=texto[i];
        char caracterClave=clave[i%longitudClave];
        int valorClave=toupper(caracterClave)-'A';
        if(isupper(letra)){
            resultado+=char((letra-'A'+valorClave)%26+'A');
        }
        else if(islower(letra)){
            resultado+=char((letra-'a'+valorClave)%26+'a');
        }
        else{
            resultado+=letra;
        }
    }
    return resultado;
}
string CifradorVigenere::descifrar(string texto){
    string resultado="";
    int longitudClave=clave.length();
    for(int i=0;i<texto.length();i++){
        char letra=texto[i];
        char caracterClave=clave[i%longitudClave];
        int valorClave=toupper(caracterClave)-'A';
        if(isupper(letra)){
            resultado+=char((letra-'A'-valorClave+26)%26+'A');
        }
        else if(islower(letra)){
            resultado+=char((letra-'a'-valorClave+26)%26+'a');
        }
        else{
            resultado+=letra;
        }
    }
    return resultado;
}

```

Ambas funciones comparten bastante código en común, pero el flujo difiere como se menciona en la fórmula. Sumamos y restamos “A” y “a” para quitarle el valor ASCII y trabajar con números en el rango del alfabeto, y se los volvemos a sumar para devolverle ese valor ASCII para mostrar el mensaje cifrado/descifrado.

Generalización a bytes (para archivos e imágenes)

Una imagen no es texto, es una fila de números (bytes) que van del 0 al 255, sin ningún patrón de letras. Si se abre de la forma normal, la computadora puede cambiar algunos de esos números sin querer y la imagen queda dañada.

Por eso se debe abrir en modo binario, para que ningún byte se altere, y leer/escribir el archivo completo de una sola vez en vez de por partes como se hace con texto.

Como no se sabe de antemano cuánto pesa el archivo, primero se calcula su tamaño, y con eso se reserva justo el espacio necesario en memoria (memoria dinámica) para guardarlo mientras se procesa.

```
void GestorArchivos::cifrarArchivo(Cifrador* cifrador, string rutaEntrada, string rutaSalida){
    ifstream entrada(rutaEntrada, ios::binary);
    if(!entrada){
        Utilidades::colorTexto(12);
        cout<<"No se pudo abrir el archivo de entrada\n"<<endl;
        Utilidades::colorTexto(7);
        return;
    }
    entrada.seekg(0, ios::end);
    long size=entrada.tellg();
    entrada.seekg(0, ios::beg);
    unsigned char* buffer=new unsigned char[size];
    entrada.read((char*)buffer, size);
    entrada.close();
    unsigned char* resultado=new unsigned char[size];
    for(long i=0; i<size; i++){
        resultado[i]=(unsigned char)cifrador->cifrarByte(buffer[i], i);
    }
    ofstream salida(rutaSalida, ios::binary);
    if(!salida){
        Utilidades::colorTexto(12);
        cout<<"No se pudo crear el archivo, verifique la ruta"<<endl;
        delete[] buffer;
        delete[] resultado;
        return;
    }
    salida.write((char*)resultado, size);
    salida.close();
    delete[] buffer;
    delete[] resultado;
    Utilidades::colorTexto(10);
    cout<<"Archivo cifrado: "<<size<<" bytes"<<endl;
    Utilidades::colorTexto(7);
}
```

Se hicieron validaciones para que el programa no trone al momento de trabajar con archivos. Validando que no se cancelaran las operaciones (abrir/guardar) y haciendo el correcto proceso de lectura (para saber el tamaño y contenido de los archivos si son archivos de texto para mostrar su contenido posterior al cifrado) y el correcto proceso de escritura para cifrar/descifrar.

```
string Utilidades::hexAtexto(string hexTexto){
    string resultado="";
    stringstream flujo(hexTexto);
    string par;
    while(flujo>>par){
        int valor=stoi(par, nullptr, 16);
        resultado+=(char)valor;
    }
    return resultado;
}
bool Utilidades::esArchivoTexto(string ruta){
    if(ruta.length()<4){
        return false;
    }
    string extension=ruta.substr(ruta.length()-4);
    for(int i=0; i<extension.length(); i++){
        extension[i]=tolower(extension[i]);
    }
    return extension==".txt";
}
```

Estas funciones son fundamentales para mostrar mensajes del cifrado/descifrado de archivos y del algoritmo XOR debido a que se trabaja con bytes, y dichos bytes pueden ser vacíos o caracteres no legibles se convierten a hexadecimal. Para archivos se extrae el mensaje y se muestra en pantalla antes de cifrar/descifrar.

Elementos de investigación propia

1. Librería <ctime>

Esta librería es utilizada dentro de Utilidades::obtenerFechaHora() con el fin de poder registrar la fecha y hora exacta en la cual se realizó cada operación.

```
string Utilidades::obtenerFechaHora(){
    time_t ahora=time(0);
    tm* horaLocal=localtime(&ahora);
    char buffer[20];
    strftime(buffer,sizeof(buffer),"%Y-%m-%d %H:%M:%S",horaLocal);
    return string(buffer);
}
```

2. Selección de archivos mediante funciones nativas del sistema

Se integraron GetOpenFileNameA y GetSaveFileNameA para que la selección de archivos sea mediante el explorador, evitando confusiones para los usuarios con las rutas de dichos archivos. (Windows)

```
string Utilidades::seleccionarArchivo(){
#ifdef _WIN32
    char rutaSeleccionada[260]="";
    OPENFILENAMEA ventana;
    ZeroMemory(&ventana,sizeof(ventana));
    ventana.lStructSize=sizeof(ventana);
    ventana.hwndOwner=NULL;
    ventana.lpstrFile=rutaSeleccionada;
    ventana.nMaxFile=260;
    ventana.lpstrFilter="Todos los archivos\0*.*\0";
    ventana.nFilterIndex=1;
    ventana.Flags=OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;
    if(GetOpenFileNameA(&ventana)){
        return string(rutaSeleccionada);
    }
    return "";
#else
    string ruta;
    cout<<"Ingrese la ruta del archivo: ";
    getline(cin,ruta);
    return ruta;
#endif
}
```

```
string Utilidades::guardarArchivoComo(){
#ifdef _WIN32
    char rutaSeleccionada[260]="";
    OPENFILENAMEA ventana;
    ZeroMemory(&ventana,sizeof(ventana));
    ventana.lStructSize=sizeof(ventana);
    ventana.hwndOwner=NULL;
    ventana.lpstrFile=rutaSeleccionada;
    ventana.nMaxFile=260;
    ventana.lpstrFilter="Todos los archivos\0*.*\0";
    ventana.nFilterIndex=1;
    ventana.Flags=OFN_OVERWRITEPROMPT;
    if(GetSaveFileNameA(&ventana)){
        return string(rutaSeleccionada);
    }
    return "";
#else
    string ruta;
    cout<<"Ingrese la ruta del archivo: ";
    getline(cin,ruta);
    return ruta;
#endif
}
```

En Windows, ambos códigos hacen que el explorador nativo se abra para poder seleccionar un archivo a cifrar/descifrar y seleccionar donde queremos que se guarde. En otros sistemas operativos se utiliza una alternativa mediante ingreso de la ruta por consola.

3. Personalización de consola

Para esta parte apliqué los conceptos que fueron aprendidos en clase (SetOutputCP(65001) en Windows) y la creación de una función colorTexto(int color), que permite manejar los colores de

los textos dentro de la consola. Para otros sistemas operativos se utilizan códigos ANSI equivalentes.

```
void Utilidades::colorTexto(int color){
#ifdef _WIN32
    HANDLE consola=GetStdHandle(STD_OUTPUT_HANDLE);
    SetConsoleTextAttribute(consola,color);
#else
    switch(color){
        case 10:
            cout<<"\033[32m";
            break;
        case 12:
            cout<<"\033[31m";
            break;
        case 14:
            cout<<"\033[33m";
            break;
        case 11:
            cout<<"\033[36m";
            break;
        default:
            cout<<"\033[0m";
            break;
    }
#endif
}
```

4. Manejo de archivos binarios

Forma correcta de usar `ios::binary`, funciones `read()/write()` y cómo manejar los punteros dentro de archivos para obtener su tamaño exacto.

```
ifstream entrada(rutaEntrada,ios::binary);
if(!entrada){
    Utilidades::colorTexto(12);
    cout<<"No se pudo abrir el archivo de entrada\n"<<endl;
    Utilidades::colorTexto(7);
    return;
}
entrada.seekg(0,ios::end);
long size=entrada.tellg();
entrada.seekg(0,ios::beg);
unsigned char* buffer=new unsigned char[size];
entrada.read((char*)buffer,size);
entrada.close();
```

5. Compilación estática (static)

Configuración de CMake para vincular estáticamente el runtime de C++ y lograr independizar el ejecutable.

```
target_link_options(CriptoManager PRIVATE -static -static-libgcc -static-libstdc++)
target_link_libraries(CriptoManager comdlg32)
```

6. Funciones virtuales

Aparte de los conocimientos brindados en clase decidí investigar su funcionamiento y como aplicarlas al proyecto. Las funciones son virtuales ya que le permiten a la clase `GestorArchivos`

poder cifrar/descifrar un archivo sin saber de antemano que algoritmo eligió el usuario. Virtual permite evitar repetir código condicional al momento de escoger que se va a cifrar un archivo.

```
public:
    Cifrador(string nombre);
    virtual ~Cifrador();
    virtual string cifrar(string texto)=0;
    virtual string descifrar(string texto)=0;
    string getNombreAlgoritmo();
    virtual char cifrarByte(unsigned char byte,int posicion)=0;
    virtual char descifrarByte(unsigned char byte,int posicion)=0;
};
```

Manual Técnico

El proyecto, tal como fue solicitado en los requisitos a cumplir, fue estructurado bajo los principios de la Programación Orientada a Objetos, separando los archivos en .h/.cpp.

Arquitectura de clases y principios de POO

Abstracción (clase Cifrador)

La clase base Cifrador es la que establece una interfaz pública mediante métodos virtuales puros (=0). Al poseer estos métodos sin implementación directa, es definida como una clase abstracta pura, que le sirve como plantilla a todas las demás clases que cifran datos.

```
#ifndef CIFRADOR_H
#define CIFRADOR_H
#include <string>
using namespace std;
class Cifrador {
protected:
    string nombreAlgoritmo;
public:
    Cifrador(string nombre);
    virtual ~Cifrador();
    virtual string cifrar(string texto)=0;
    virtual string descifrar(string texto)=0;
    string getNombreAlgoritmo();
    virtual char cifrarByte(unsigned char byte,int posicion)=0;
    virtual char descifrarByte(unsigned char byte,int posicion)=0;
};
#endif
```

Uso de modificador protected

El atributo nombreAlgoritmo se define como protected dentro de la clase Cifrador.h a razón de que esto le permite a las clases hijas (CifradorCesar, CifradorXOR, CifradorVigenere) pueden acceder y

asignar directamente este atributo en sus constructores sin que este expuesto de manera pública al exterior de las clases deseadas.

```
#ifndef CIFRADOR_H
#define CIFRADOR_H
#include <string>
using namespace std;
class Cifrador {
protected:
    string nombreAlgoritmo;
```

Herencia encapsulamiento

y

Las tres clases hijas (CifradorCesar, CifradorXOR, CifradorVigenere) heredan públicamente de Cifrador. Cada clase encapsula sus claves de forma privada, protegiendo el estado criptográfico del objeto.

```
#ifndef CIFRADORCESAR_H
#define CIFRADORCESAR_H
#include "Cifrador.h"
class CifradorCesar:public Cifrador
{
private:
    int clave;
public:
    CifradorCesar(int claveDesplazamiento);
    ~CifradorCesar();
    string cifrar(string texto);
    string descifrar(string texto);
    char cifrarByte(unsigned char byte,int posicion);
    char descifrarByte(unsigned char byte,int posicion);
};
#endif // CIFRADORCESAR_H
```

Polimorfismo

Dentro del programa se cuenta con un puntero Cifrador* miCifrador el cual permite manipular cualquier algoritmo en tiempo de ejecución. En GestorArchivos y en el main la llamada a cifrar/descifrarByte() se resuelve de forma dinámica según la subclase real que se instancia en el Heap.

Uso técnico de punteros dentro del sistema

1. Buffer para procesamiento de archivos (unsigned char* buffer)

Dentro de GestorArchivos y Utilidades::copiarBinario, el puntero mencionado apunta al bloque contiguo de memoria RAM en el Heap donde reside la totalidad del archivo binario durante el cifrado, lectura y escritura. Se usa unsigned para evitar los valores negativos dentro del puntero.

```
unsigned char* buffer=new unsigned char[size];
```

2. Puntero polimórfico (Cifrador* miCifrador)

Este puntero permite referenciar indistintamente cualquier instancia de CifradorCesar, CifradorXOR o CifradorVigenere en tiempo de ejecución. El puntero no conoce el algoritmo específico al momento de compilación.

```
Cifrador* miCifrador=new CifradorCesar(clave);
```

3. Doble puntero (Usuario** listaUsuarios)

Este puntero es utilizado dentro de GestorUsuarios para administrar un arreglo dinámico de memoria que apuntan a objetos de tipo Usuario en el Heap. Con este puntero es que podemos modificar la lista y hacer todas las operaciones deseadas dentro de ella.

```
Usuario** listaUsuarios;
```

Optimización de memoria

1. Arreglos dinámicos “inteligentes”

Dentro de GestorUsuarios e HistorialOperaciones se cuenta con una función llamada redimensionar(). Dicha función está destinada a, como lo dice su nombre, redimensionar los arreglos dinámicos. Decidí que, en lugar de asignar bloques masivos de memoria estática, las clases mencionadas podrían iniciar con una capacidad reducida (5 elementos). Al alcanzar el límite establecido, la función privada ya mencionada se encarga de duplicar ese espacio y reasignar el arreglo. Esto garantiza un consumo de RAM proporcional al uso que el usuario vaya usando.

```
void HistorialOperaciones::redimensionar(){
    int nuevaCapacidad=capacidad*2;
    Operacion* listaNueva=new Operacion[nuevaCapacidad];
    for(int i=0;i<cantOperaciones;i++){
        listaNueva[i]=listaOperaciones[i];
    }
    delete[] listaOperaciones;
    listaOperaciones=listaNueva;
    capacidad=nuevaCapacidad;
}
```

2. Reserva exacta de bytes

En GestorArchivos antes de reservar memoria para leer un archivo, se posiciona el puntero de lectura al final con seekg(0,ios::end)) y se consulta el tamaño real/exacto con tellg(). Esto permite hacer new unsigned char[size] con la cantidad exacta de bytes, evitando que se desperdicie memoria.

```
entrada.seekg(0,ios::end);
long size=entrada.tellg();
entrada.seekg(0,ios::beg);
unsigned char* buffer=new unsigned char[size];
entrada.read((char*)buffer,size);
entrada.close();
```

Módulo de historial, auditoría y ordenamiento (Nivel 4)

El nivel 4 es completado e implementado mediante las clases Operación y la clase gestora HistorialOperaciones. Funcionan de la siguiente manera:

Entidad Operacion

Esta clase encapsula el nombre del usuario, el tipo de operación (cifrado, descifrado o copia), el algoritmo utilizado, el nombre del archivo/origen (si fue en consola) y la fecha y hora.

```
#ifndef OPERACION_H
#define OPERACION_H
#include <string>
using namespace std;
class Operacion
{
private:
    string usuario;
    string tipoOperacion;
    string algoritmo;
    string archivo;
    string fechaHora;
public:
    Operacion();
    Operacion(string _usuario, string _tipoOperacion, string _algoritmo, string _archivo, string _fechaHora);
    ~Operacion();
    string getUsuario();
    string getTipoOperacion();
    string getAlgoritmo();
    string getArchivo();
    string getFechaHora();
};
#endif // OPERACION_H
```

Algoritmo de ordenamiento por fecha (Burbuja)

La clase HistorialOperaciones implementa el ordenamiento burbuja sobre el arreglo dinámico listaOperaciones ordenando los datos por fecha y hora en el cual se realizaron.

```
void HistorialOperaciones::ordenarPorFecha(){
    for(int i=0; i<cantOperaciones-1; i++){
        for(int j=0; j<cantOperaciones-1-i; j++){
            if(listaOperaciones[j].getFechaHora()>listaOperaciones[j+1].getFechaHora()){
                Operacion temp=listaOperaciones[j];
                listaOperaciones[j]=listaOperaciones[j+1];
                listaOperaciones[j+1]=temp;
            }
        }
    }
}
```

Implementación de búsqueda lineal

Requisito cumplido en la clase HistorialOperaciones. Al usuario se le permite buscar las operaciones realizadas y guardadas en el sistema por el nombre de usuario.

```
void HistorialOperaciones::buscarPorUsuario(string usuario){
    bool encontrado=false;
    cout<<left<<setw(20)<<"Fecha y Hora"<<setw(12)<<"Tipo"<<setw(12)<<"Algoritmo"<<"Archivo"<<endl;
    cout<<string(78, '-')<<endl;
    for(int i=0; i<cantOperaciones; i++){
        if(listaOperaciones[i].getUsuario()==usuario){
            cout<<left<<setw(20)<<listaOperaciones[i].getFechaHora()<<setw(12)<<listaOperaciones[i].getTipoOperacion()<<setw(12)<<listaOperaciones[i].getAlgoritmo()<<listaOperaciones[i].getArc
            encontrado=true;
        }
    }
    if(!encontrado){
        Utilidades::colorTexto(12);
        cout<<"No se encontraron operaciones para ese usuario\n"<<endl;
        Utilidades::colorTexto(7);
    }
}
```

Pruebas y resultados

Durante el desarrollo se realizaron pruebas constantes con datos válidos e inválidos, verificando tanto el funcionamiento correcto del sistema como el manejo de errores.

Pruebas con datos inválidos

Se comprobó que el sistema de registro de usuarios rechaza de forma correcta los nombres de usuario ya existentes y las credenciales vacías, mostrando el mensaje de error hasta que se llenen los datos correctamente.

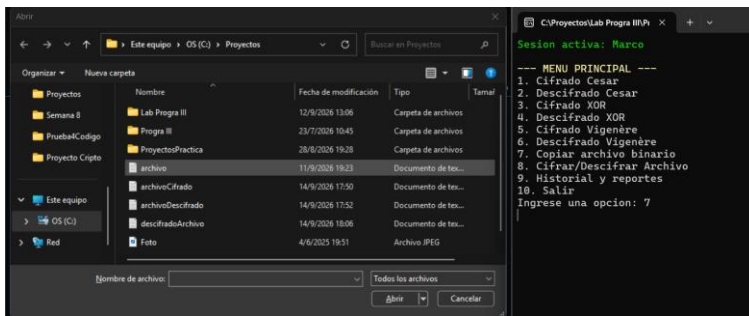
```
--- Proyecto: Cripto-Manager Cifrador y Descifrador de archivos ---  
Bienvenido al sistema de ingreso  
1. Iniciar sesión  
2. Registrarme  
3. Salir  
Opción: 2  
Usuario:  
El texto no puede estar vacío  
  
Usuario: Marco  
Contraseña: a  
Ese nombre de usuario existe
```

También se validaron que al momento de cifrar/descifrar archivos si se decide no seleccionar ninguno se muestre un mensaje de error y se cancele la operación.

```
C:\Proyectos\Lab Progra III\Pr... x + v  
Sesion activa: Marco  
  
--- MENU PRINCIPAL ---  
1. Cifrado Cesar  
2. Descifrado Cesar  
3. Cifrado XOR  
4. Descifrado XOR  
5. Cifrado Vigenère  
6. Descifrado Vigenère  
7. Copiar archivo binario  
8. Cifrar/Descifrar Archivo  
9. Historial y reportes  
10. Salir  
Ingrese una opcion: 8  
No se seleccionó ningún archivo  
  
Presione ENTER para continuar...  
|
```

Prueba con datos válidos

Se verifico el flujo correcto para todos los niveles de cifrado/descifrado tanto texto plano en consola como archivos (texto e imágenes). En Windows, los archivos se seleccionan mediante el explorador nativo del sistema. En otros sistemas operativos, se solicita la ruta del archivo mediante consola.



```
C:\Proyectos\Lab Progra III\Pr...
Sesion activa: Marco

--- MENU PRINCIPAL ---
1. Cifrado Cesar
2. Descifrado Cesar
3. Cifrado XOR
4. Descifrado XOR
5. Cifrado Vigenere
6. Descifrado Vigenere
7. Copiar archivo binario
8. Cifrar/Descifrar Archivo
9. Historial y reportes
10. Salir
Ingrese una opcion: 7

Archivo copiado: 76674 bytes

Presione ENTER para continuar...
```

```
Sesion activa: Marco

--- MENU PRINCIPAL ---
1. Cifrado Cesar
2. Descifrado Cesar
3. Cifrado XOR
4. Descifrado XOR
5. Cifrado Vigenere
6. Descifrado Vigenere
7. Copiar archivo binario
8. Cifrar/Descifrar Archivo
9. Historial y reportes
10. Salir
Ingrese una opcion: 1
Ingrese el mensaje: Hola mundo
Ingrese la clave (1-26): 3
Mensaje cifrado: Krod pxqgr
```

Evidencia de validaciones y del historial

El historial de operaciones se muestra tabulado con cada acción realizada, confirmando el correcto registro de fecha/hora, usuario, algoritmo y archivo procesado (consola o ruta).

```
void HistorialOpera
bool encontrado
cout<<left<<set
cout<<string(7)
for(int i=1;i<=
if(!listaOpe
cout<<endl
}
}
if(!encontrado)
Utilidades:
cout<<"No s
Utilidades:
}
```

Resultado final obtenido

El sistema cumple con todos los requisitos, incluidos los cuatro niveles solicitados, mostrando de forma clara en consola cada resultado (cifrado, descifrado, reportes, historial), gestionando errores evitando que el programa colapse de forma inesperada y persistiendo tanto los usuarios como el historial de operaciones entre otras distintas ejecuciones del programa.

Manual de Usuario

Introducción y propósito del sistema

El programa desarrollado, Cripto-Manager, es un sistema de cifrado y descifrado de archivos/texto multinivel desarrollado en C++. El sistema permite asegurar información confidencial procesando tanto textos planos en consola como archivos binarios completos de cualquier extensión (jpg, png, txt, etc).

El proyecto cumple con los 4 niveles estipulados en la asignación brindada, implementando los cifrados/descifrados César, XOR y Vigenère, generalización a nivel de bytes (0-255), interacción con el explorador nativo de Windows, gestión de usuarios, historial de movimientos con fecha y hora reales, reportes, ordenamientos y búsquedas.

Requisitos de ejecución y guía de inicio rápido

- Entorno: El proyecto esta preparado para compilarse en Windows, Linux y macOS. Algunas funciones específicas del sistema operativo utilizan implementaciones diferentes mediante directivas de compilación, manteniendo el funcionamiento principal del sistema.
- Consola interactiva: La consola está configurada con soporte UTF-8 que soportan tildes y la letra ñ. Además de que permite la personalización con colores para títulos, errores, salidas esperadas, etc.
- Portabilidad estática: En Windows, el ejecutable compila estáticamente con las banderas -static, -static-libgcc y -static-libstdc++, reduciendo la dependencia de librerías externas del runtime en C++.

Guía de pasos de inicio rápido

1. Ejecutar el programa
2. En la pantalla de ingreso, seleccione la opción 2 para crear un usuario o bien la opción 1 si ya cuenta con un registro
3. Tras identificarse presione ENTER para desplegar el menú principal
4. Use el sistema a su gusto, al finalizar seleccione la opción 10, los datos del sistema se guardaran de forma automática al cierre de la aplicación

```
--- Proyecto: Cripto-Manager Cifrador y Descifrador de archivos ---  
Bienvenido al sistema de ingreso  
1. Iniciar sesión  
2. Registrarme  
3. Salir  
Opción: |
```

Control de acceso y gestión de usuarios

El sistema cuenta con un el módulo GestorUsuarios con almacenamiento permanente en el archivo de texto llamado usuarios.

El sistema valida que el usuario no este repetido y que las credenciales sean correctas al momento de iniciar sesión. En caso de tener un error o algún fallo el sistema mostrará los errores en color rojo. Las acciones (como la salida de los mensajes de cifrado/descifrado) serán mostradas en pantalla en color verde.

```
--- Proyecto: Cripto-Manager Cifrador y Descifrador de archivos ---  
Bienvenido al sistema de ingreso  
1. Iniciar sesión  
2. Registrarme  
3. Salir  
Opción: |
```

Inicio de sesión (opción 1): Si las credenciales son válidas se despliega el menú principal mostrando la sesión activa en todo momento junto con las 10 opciones de control.

```
Sesion activa: Marco  
  
--- MENU PRINCIPAL ---  
1. Cifrado Cesar  
2. Descifrado Cesar  
3. Cifrado XOR  
4. Descifrado XOR  
5. Cifrado Vigenere  
6. Descifrado Vigenere  
7. Copiar archivo binario  
8. Cifrar Archivo  
9. Historial y reportes  
10. Salir  
Ingrese una opcion: |
```

Registro (opción 2): Si el nombre de usuario es existente o si las credenciales se ingresan vacías se mostrara un mensaje de error.

```
--- Proyecto: Cripto-Manager Cifrador y Descifrador de archivos ---
Bienvenido al sistema de ingreso
1. Iniciar sesión
2. Registrarme
3. Salir
Opción: 2
Usuario:
El texto no puede estar vacío

Usuario: Marco
Contraseña: a
Ese nombre de usuario existe
```

Operaciones en consola y menú principal (opciones 1-6)

Las primeras seis opciones son los cifrados normales, texto a cifrar/descifrar en la consola directamente. Las acciones hechas en consola se registran automáticamente en el historial de operaciones:

1. Cifrado César: Recibe un mensaje y una clave entera (1-26). Funciona de la siguiente manera: Desplaza las letras correspondientes del texto las posiciones de la clave ingresada (en forma circular).
2. Descifrado César: Proceso inverso al cifrado, invierte el desplazamiento alfabético utilizando la clave especificada por el usuario.
3. Cifrado XOR: Aplica la operación bit a bit contra la clave ingresada y muestra la salida en formato hexadecimal. Se imprime en hexadecimal debido a que como se trabaja con bits hay caracteres vacíos/raros y no algo como “mensaje” fijo.
4. Descifrado XOR: Solicita una cadena en formato hexadecimal (tal como es generada al cifrar), la reconvierte en a binario y aplica el descifrado simétrico.
5. Cifrado Vigenère: Solicita un mensaje y una clave exclusivamente alfabética (validada dentro del sistema), el proceso evalúa los caracteres de la clave contra el mensaje, la cadena se desplaza el número correspondiente a la letra de la clave.
6. Descifrado Vigenère: Realiza la resta de desplazamientos alfabéticos, usando la clave, recuperando el mensaje original.

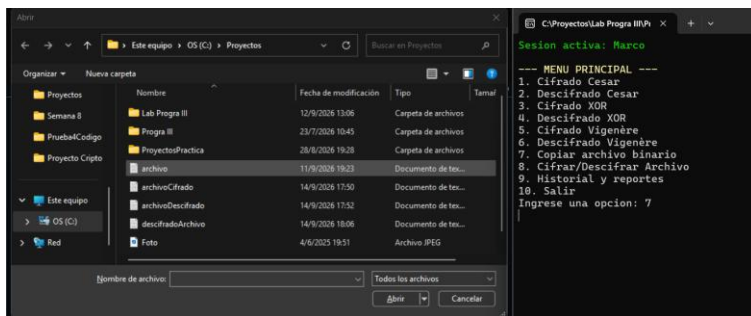
```
Sesion activa: Marco
--- MENU PRINCIPAL ---
1. Cifrado Cesar
2. Descifrado Cesar
3. Cifrado XOR
4. Descifrado XOR
5. Cifrado Vigenere
6. Descifrado Vigenere
7. Copiar archivo binario
8. Cifrar/Descifrar Archivo
9. Historial y reportes
10. Salir
Ingrese una opcion: 1
Ingrese el mensaje: Hola mundo
Ingrese la clave (1-26): 3
Mensaje cifrado: Krod pxqgr
```

```
Sesion activa: Marco
--- MENU PRINCIPAL ---
1. Cifrado Cesar
2. Descifrado Cesar
3. Cifrado XOR
4. Descifrado XOR
5. Cifrado Vigenere
6. Descifrado Vigenere
7. Copiar archivo binario
8. Cifrar/Descifrar Archivo
9. Historial y reportes
10. Salir
Ingrese una opcion: 3
Ingrese el mensaje: Hola mundo
Ingrese la clave: hola
Mensaje cifrado (hex): 20 00 00 00 48 02 19 0F 0C 00
Presione ENTER para continuar...
```

Operaciones de archivos y auditoría (opciones 7 a 9)

Opción 7: Copiar archivo binario

Esta opción comprueba la lectura y escritura limpia en modo binario (con ios::binary) mediante el selector de archivos nativo del sistema operativo. El proceso se guarda en el historial como “Copia” de tipo “Binario”. Implementado para comprobar que el sistema no corrompe los archivos en su lectura.



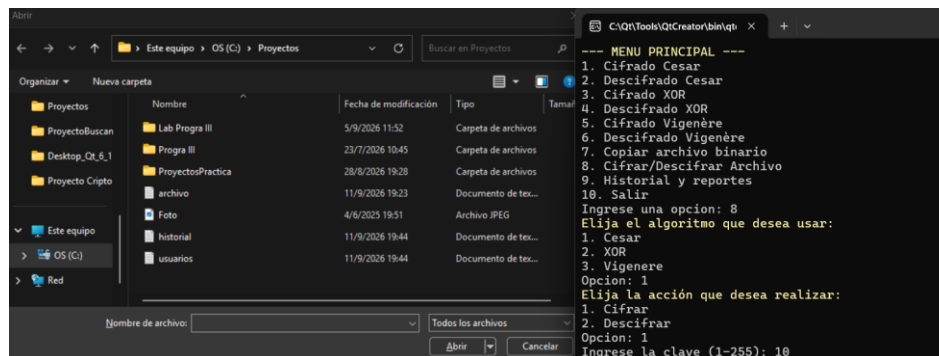
```
C:\Proyectos\Lab Progra III\Pi x + v
Sesion activa: Marco

--- MENU PRINCIPAL ---
1. Cifrado Cesar
2. Descifrado Cesar
3. Cifrado XOR
4. Descifrado XOR
5. Cifrado Vigenere
6. Descifrado Vigenere
7. Copiar archivo binario
8. Cifrar/Descifrar Archivo
9. Historial y reportes
10. Salir
Ingrese una opcion: 7
Archivo copiado: 76674 bytes

Presione ENTER para continuar...
```

Opción 8: Cifrar/Descifrar archivo

Permite cifrar o descifrar archivos con el algoritmo que el usuario seleccione. Se le solicita al usuario escoger un algoritmo, un tipo de acción y para los tres métodos ingresar una clave (para César se aceptan números del 1 al 255 por temas de binario). En Windows se abre el explorador nativo para seleccionar el archivo y la ubicación donde se guardará. En otros sistemas operativos se solicita la ruta mediante consola. La confirmación exitosa se muestra en verde, si algo falla al abrir el archivo o donde se guardara se muestra en rojo.



Opción 9: Historial y reportes (submenú)

Al escoger la opción nueve, el sistema despliega un submenú el cual permite realizar las siguientes opciones:

1. Ver historial: Muestra la tablada con formato centrado las acciones realizadas y registradas con fecha y hora, usuario, tipo, algoritmo y archivo (ruta o consola).

2. Ordenar historial por fecha: Aplica el ordenamiento burbuja sobre todos los registros guardados y los ordena según su fecha de realización.
3. Buscar operaciones por usuario: Filtra y despliega únicamente las operaciones realizadas por un usuario. Si el usuario ingresado no existe se mostrará el error en pantalla en color rojo.
4. Generar reporte de uso: Despliega las estadísticas con los conteos totales de todos los usos según tipo de cifrado/descifrado (César, XOR, Vigenère).

Al terminar de seleccionar una de las opciones, el sistema regresa al menú principal para seguir en ejecución.

```

Ingrese una opcion: 9
--- HISTORIAL Y REPORTE ---
1. Ver historial completo
2. Ordenar historial por fecha
3. Buscar operaciones por usuario
4. Generar reporte de uso
Opcion: 1

```

Fecha y hora	Usuario	Tipo	Algoritmo	Archivo
2026-09-14 18:05:22	Marco	Cifrado	Cesar	consola
2026-09-14 18:05:55	Marco	Cifrado	Cesar	C:\Proyectos\archivo.txt
2026-09-14 18:06:37	Marco	Descifrado	Cesar	C:\Proyectos\salida.txt
2026-09-18 02:05:53	Marco	Cifrado	Cesar	consola
2026-09-18 02:06:40	Marco	Cifrado	XOR	consola
2026-09-18 02:08:11	Marco	Copia	Binario	C:\Proyectos\Foto.jpeg

```

Presione ENTER para continuar...

```

Solución de problemas

- Cancelación de selección de archivo: Si al momento de seleccionar una de las opciones se cancela la selección del archivo, el programa no tronará y mostrará un mensaje en rojo indicando que la operación fue cancelada.
- Descifrado XOR de consola: Al descifrar un mensaje en XOR asegúrate de ingresarlo en forma hexadecimal (tal como te lo da en el cifrado XOR presentado en la opción 3). Ejemplo: 48 65 6C 6C 6F.
- Archivos corruptos: Asegúrate de usar la misma clave y algoritmo cuando vas a cifrar/descifrar un archivo. Modificar una clave en archivos binarios altera su formato nativo y encabezados.

```

C:\Proyectos\Lab Progra III\Pi x + v
Sesion activa: Marco
--- MENU PRINCIPAL ---
1. Cifrado Cesar
2. Descifrado Cesar
3. Cifrado XOR
4. Descifrado XOR
5. Cifrado Vigenere
6. Descifrado Vigenere
7. Copiar archivo binario
8. Cifrar/Descifrar Archivo
9. Historial y reportes
10. Salir
Ingrese una opcion: 8
No se seleccionó ningún archivo
Presione ENTER para continuar...

```

Conclusiones

1. Se logró aplicar de forma práctica los principios de la Programación Orientada a Objetos (herencia, encapsulamiento, polimorfismo y abstracción) en un sistema real, comprobando como el uso de una clase abstracta con funciones virtuales evitar la repetición de código/lógica de selección de algoritmo en distintas partes del programa.
2. Durante el desarrollo se presentaron y resolvieron distintos errores típicos de C++ como: variables sin inicializar por no reposicionar el cursor de lectura de un archivo (seekg), separadores incorrectos al leer datos de un archivo de texto, conflictos de nombres entre tipos de Windows y de la librería estándar de C++. Cada uno fue resuelto identificando la causa exacta antes de aplicar la corrección definitiva.
3. Se aprendió a generalizar un algoritmo pensado originalmente para el alfabeto (26 letras) a un algoritmo que es capaz de operar sobre cualquier byte (0-255), lo cuál permitió la correcta encriptación y desencriptación de mensajes de texto a archivos binarios completos, incluyendo claro también imágenes.
4. Se comprendió la importancia de anclar rutas de archivo a la ubicación real del ejecutable en lugar de rutas relativas al directorio de ejecución (problemas del guardado de historial y usuarios), evitando que el programa se comporte de forma distinta según cómo o desde donde se ejecute.
5. Se reforzó el manejo de memoria dinámica y punteros, incluyendo un doble puntero, entendiendo la diferencia entre copiar un objeto y copiar la dirección donde vive ese objeto, y por supuesto la responsabilidad de liberar toda la memoria reservada con new.

Bibliografías/Referencias

- De Camino Al Hacking. (2025, 3 de septiembre). *Cifrados simétricos modernos: Cifrado XOR*. [Video]. Youtube. <https://youtu.be/IIUp2rc6FYc?si=-gPCwcr1AUNy6tIB>
- Diaz, F. (2020, 5 de mayo). *Cifrado Cesar*. [Video]. Youtube. https://youtu.be/DT3bsFpuikY?si=Cgf2CP_FqBlzvRKD
- Llamas, L. (s.f). *Manejo de archivos binarios en C++*. Luis Llamas: Ingeniería, informática y diseño. <https://www.luisllamas.es/cpp-archivos-binarios/>
- Muñoz, F.A. (2010). *Historia de la criptografía y cifrados clásicos*. Universidad de Granada. <https://www.ugr.es/~anillos/textos/pdf/2010/EXPO-1.Criptografia/02a04.htm>
- StudySmarter. (s.f.). *Cifrados poli alfabéticos*. StudySmarter. <https://www.studysmarter.es/resumenes/ingenieria/ingenieria-de-telecomunicaciones-ingenieria/cifrados-polialfabeticos/>
- VM, D. (2020, 2 de junio). *Criptografía. Método Vigenere* [Video]. Youtube. <https://youtu.be/ZqPCLJUwamE?si=3Gu9pSaakc7y-z4o>
- Wikipedia. (s.f.). *Cifrado XOR*. En Wikipedia. https://es.wikipedia.org/wiki/Cifrado_XOR

Anexos

Se incluye el código fuente completo en la carpeta llamada: Código Fuente del archivo comprimido de entrega, con todos los archivos .h y .cpp orgnizados por clase, tal como es solicitado en la guía del proyecto.

El ejecutable compilado (CriptoManager.exe) se incluye en la carpeta Ejecutable listo para correr sin necesidad de un editor de código.